GA GABON FÉRIÉS

Calendrier Interactif des Jours Fériés
de la République Gabonaise

BRIEF PROJET · CAHIER DES CHARGES · SCHÉMA BASE DE DONNÉES

Statut	**Open Source · v1.0**
Stack	Next.js · Supabase · Vercel
Date	23 mars 2026

01 — BRIEF PROJET

1.1 Contexte & Problématique

Au Gabon, l'information sur les jours fériés reste fragmentée : elle circule principalement via des communiqués officiels diffusés à la télévision nationale ou des recherches Google répétitives. Il n'existe aucune plateforme centralisée, interactive et communautaire permettant aux citoyens d'anticiper les jours chômés payés et de s'approprier l'histoire et la culture liées à ces dates.

GABON FÉRIÉS répond à ce vide en proposant une plateforme web open source combinant : utilité immédiate (agenda), éducation (histoire), gamification (badges, quiz) et viralité (partage social).

1.2 Vision du Produit

« Faire du calendrier des jours fériés gabonais la référence numérique nationale — un outil que chaque Gabonais consulte par réflexe, partage avec fierté et qui renforce le lien civique à l'histoire du pays. »

1.3 Objectifs SMART

#	Objectif	Indicateur de succès
O1	Informer	100 % des jours fériés officiels disponibles au lancement
O2	Engager	Taux de complétion quiz ≥ 40 % à M+3
O3	Croître	1 000 utilisateurs enregistrés à M+6 (sans budget pub)
O4	Fiabiliser	Délai moyen de validation admin ≤ 24h après soumission

1.4 Cibles Utilisateurs

Persona 1 — Le Salarié Gabonais

- Âge : 25–45 ans, secteur privé ou public
- Besoin : savoir quand poser des congés, anticiper les ponts
- Canal préféré : WhatsApp, Facebook

Persona 2 — L'Étudiant Curieux

- Âge : 18–25 ans, universite ou lycée
- Besoin : apprendre l'histoire du Gabon de façon ludique
- Canal préféré : TikTok, Instagram

Persona 3 — L'Administrateur / Modérateur

- Rôle interne : vérifie et publie les nouvelles dates
- Besoin : interface simple, sans code, efficace sous 2 minutes

1.5 Périmètre du Projet (MVP)

☑ DANS LE SCOPE (MVP)	⊘ HORS SCOPE (V2+)
Calendrier annuel des jours fériés	Application mobile native iOS/Android
Soumission citoyenne + workflow de validation	Intégration API gouvernementale officielle
Système de badges et quiz par date	Traduction en langues locales (fang, myene…)
Notifications Push/Email (J-1, ponts)	Monétisation / version Premium
Authentification (email + Google OAuth)	Intégration Slack / Teams native
Export iCal / Google Calendar	Tableau de bord analytics avancé
Interface d'administration (CMS)	IA pour génération de contenu culturel

02 — CAHIER DES CHARGES FONCTIONNEL

2.1 Architecture Globale

Couche	Technologie
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind CSS
Backend / BDD	Supabase (PostgreSQL + Auth + Realtime + Storage + Edge Functions)
Notifications	Supabase Edge Functions + Resend (email) + OneSignal (push web)
Hébergement	Vercel (frontend) + Supabase Cloud (backend)
CI/CD	GitHub Actions + Vercel Preview Deployments
Monitoring	Sentry (erreurs) + Supabase Dashboard (analytics)

2.2 Modules Fonctionnels

MODULE A — Calendrier Public

- Affichage mensuel et annuel des jours fériés avec code couleur (Vert = férié fixe, Jaune = mobile, Bleu = pont potentiel)
- Carte interactive par date : nom officiel, histoire, traditions, galerie photos
- Cases « Mystère » : verrouillées jusqu'au jour J, débloquées par un clic
- Widget « Prochain pont » avec compte à rebours partageable
- Bouton export : iCal / Google Calendar / Outlook (en un clic)

MODULE B — Authentification & Profil

- Inscription / connexion : email+mot de passe ou Google OAuth (via Supabase Auth)
- Profil utilisateur : avatar personnalisable (débloqué via points), pseudo, département gabonais
- Tableau de bord personnel : badges obtenus, points cumulés, quiz complétés, historique de soumissions

MODULE C — Soumission Citoyenne

- Formulaire de soumission : titre, date, type (férié / chômé / pont), description, source officielle (URL obligatoire)
- Statut de soumission : En attente → En vérification → Approuvé / Rejeté
- Notification utilisateur (email + push) à chaque changement de statut
- Obligation de source : le champ URL source est requis et validé côté serveur

MODULE D — Gamification

- Barre de Patriotisme : jauge globale progressant à chaque consultation de fiche historique
- Badges : débloqués automatiquement selon les actions (voir tableau 2.3)
- Série de connexion : bonus mensuel si consultation du calendrier chaque mois

- Quiz : 3 questions par jour férié (QCM, Vrai/Faux) avec points virtuels
- Classement (Leaderboard) : top utilisateurs du mois, partage du score sur WhatsApp

MODULE E — Notifications

- Rappel J-1 : email + push la veille de chaque jour férié
- Alerte Pont : notification spéciale si le férié crée un pont (mardi/jeudi)
- Appel Quiz : notification 7 jours avant chaque date nationale importante
- Félicitations : push automatique lors de l'obtention d'un badge
- Le Saviez-vous ? : notification culturelle courte le matin du jour J
- Paramétrage utilisateur : choix des canaux et types de notifications reçus

MODULE F — Administration (CMS)

- Tableau de bord : compteurs (utilisateurs, soumissions en attente, quiz complétés)
- Gestion du calendrier : CRUD dates, activation/désactivation des ponts, calcul automatique fêtes mobiles
- File de validation : liste des soumissions à examiner, avec sources vérifiables en un clic, boutons Approuver / Rejeter + commentaire
- Gestion Gamification : banque de questions quiz, CRUD badges, seuils de points
- Gestion Notifications : éditeur de messages, segmentation audience, planification
- Gestion Utilisateurs : liste, rôles, modération

2.3 Catalogue des Badges

Badge	Condition de débloquage	Points requis	Icône
Historien	Lire les fiches du 17 août ET du 30 août	— (action)	
Patriote	Consulter toutes les fêtes nationales d'août	— (action)	
Culturel	Lire 3 fiches de fêtes religieuses	— (action)	
Quiz Master	Compléter 10 quiz avec score ≥ 80 %	500 pts	
Ambassadeur	Partager 3 invitations WhatsApp acceptées	— (parrainage)	
Assidu	Consulter le calendrier 3 mois consécutifs	— (série)	
Contributeur	Avoir une soumission approuvée	— (action)	

2.4 Jours Fériés Officiels (Données Initiales)

Date	Intitulé officiel	Catégorie	Type
1er janvier	Jour de l'An	Fête Civile	Fixe

12 mars	Journée de la Rénovation	Fête Civile	Fixe
17 avril	Journée de la Femme Gabonaise	Fête Civile	Fixe
1er mai	Fête du Travail	Fête Civile	Fixe
15 août	Assomption	Fête Religieuse	Fixe
16–17 août	Fête de l'Indépendance	Fête Nationale	Fixe
30 août	Journée Nationale de la Libération	Fête Nationale	Fixe
1er novembre	Toussaint	Fête Religieuse	Fixe
25 décembre	Noël	Fête Religieuse	Fixe
Variable	Lundi de Pâques	Fête Religieuse	Mobile (calendrier chrétien)
Variable	Ascension	Fête Religieuse	Mobile (J+39 après Pâques)
Variable	Lundi de Pentecôte	Fête Religieuse	Mobile (J+49 après Pâques)
Variable	Aïd al-Fitr (Korité)	Fête Religieuse	Mobile (calendrier lunaire)
Variable	Aïd al-Adha (Tabaski)	Fête Religieuse	Mobile (calendrier lunaire)

2.5 Exigences Non-Fonctionnelles

Performance

- Temps de chargement initial < 2 secondes (LCP)
- Score Lighthouse ≥ 85 sur mobile
- Mise à jour temps réel (Supabase Realtime) < 500 ms après approbation admin

Sécurité

- Authentification JWT via Supabase Auth
- Row Level Security (RLS) sur toutes les tables sensibles
- Validation des URLs sources côté serveur (Edge Function) avant insertion
- Rate limiting sur les soumissions : max 5 par utilisateur par 24h

Accessibilité & Responsive

- Design mobile-first (cible principale : smartphones, réseau 3G)
- Support navigateurs : Chrome, Firefox, Safari, Samsung Internet
- WCAG 2.1 AA minimum

03 — SCHÉMA BASE DE DONNÉES SUPABASE

3.1 Vue d'Ensemble

Le schéma utilise PostgreSQL via Supabase. Toutes les tables sont protégées par des politiques Row Level Security (RLS). Les triggers gèrent automatiquement updated_at, les points et le déclenchement des notifications.

Table	Rôle
profiles	Extension de auth.users — données publiques du profil utilisateur
holidays	Référentiel central des jours fériés validés
holiday_submissions	Soumissions citoyennes en attente de validation
holiday_content	Contenu éditorial enrichi (histoire, traditions, médias) par date
quizzes	Quiz associés à chaque jour férié
quiz_questions	Questions et choix de réponses
quiz_attempts	Tentatives et scores des utilisateurs
badges	Catalogue des badges disponibles
user_badges	Table de jonction : badges obtenus par utilisateur
notifications	File de notifications planifiées et envoyées
notification_preferences	Préférences de notification par utilisateur
user_streaks	Suivi des séries de connexion mensuelles
audit_logs	Journal d'audit des actions admin

3.2 SQL Complet — Création des Tables

Copiez ce bloc SQL dans l'éditeur Supabase (Dashboard → SQL Editor) pour initialiser la base de données.

A. Activation des Extensions

```
-- Extensions requises
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pg_cron";
CREATE EXTENSION IF NOT EXISTS "unaccent";
```

B. Table : profiles

```
-- ═══════════════════════════════════════════════
-- TABLE: profiles (extension de auth.users)
-- ═══════════════════════════════════════════════
CREATE TABLE public.profiles (
  id           UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  username     TEXT UNIQUE NOT NULL CHECK (char_length(username) BETWEEN 3 AND 30),
  full_name    TEXT,
  avatar_url   TEXT,
```

```
  department   TEXT,           -- Province/département gabonais
  role         TEXT NOT NULL DEFAULT 'user'
                 CHECK (role IN ('user', 'editor', 'moderator', 'admin')),
  total_points INTEGER NOT NULL DEFAULT 0 CHECK (total_points >= 0),
  is_active    BOOLEAN NOT NULL DEFAULT true,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Index pour le leaderboard
CREATE INDEX idx_profiles_points ON public.profiles(total_points DESC);
CREATE INDEX idx_profiles_role   ON public.profiles(role);

-- Trigger: sync auto depuis auth.users à la création
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, username, full_name, avatar_url)
  VALUES (
    NEW.id,
    COALESCE(NEW.raw_user_meta_data->>'username', split_part(NEW.email, '@', 1)),
    NEW.raw_user_meta_data->>'full_name',
    NEW.raw_user_meta_data->>'avatar_url'
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

C. Table : holidays

```
-- ==================================================
-- TABLE: holidays (référentiel des jours fériés)
-- ==================================================
CREATE TYPE holiday_category AS ENUM (
  'civil', 'national', 'religious_christian', 'religious_muslim', 'other'
);
CREATE TYPE holiday_recurrence AS ENUM ('fixed', 'mobile_christian', 'mobile_lunar',
'decree');

CREATE TABLE public.holidays (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name            TEXT NOT NULL,
  name_short      TEXT,
  description     TEXT,
  category        holiday_category NOT NULL,
  recurrence      holiday_recurrence NOT NULL DEFAULT 'fixed',
  -- Pour les fêtes fixes : mois + jour
  fixed_month     SMALLINT CHECK (fixed_month BETWEEN 1 AND 12),
  fixed_day       SMALLINT CHECK (fixed_day BETWEEN 1 AND 31),
  -- Pour les fêtes mobiles : date calculée ou saisie pour l'année en cours
  date_year       SMALLINT,          -- ex: 2025
  date_value      DATE,              -- date réelle pour l'année courante
  -- Pont : si le férié crée un pont (ex: mardi/jeudi)
  is_bridge       BOOLEAN DEFAULT false,
  bridge_date     DATE,              -- date du pont accordé
  -- Statut
  is_published    BOOLEAN NOT NULL DEFAULT false,
  is_paid         BOOLEAN NOT NULL DEFAULT true,  -- chômé payé ?
  -- Métadonnées
  official_source TEXT,
  color_hex       CHAR(6) DEFAULT '009E60',
  display_order   SMALLINT DEFAULT 0,
  created_by      UUID REFERENCES public.profiles(id),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_holidays_date   ON public.holidays(date_value);
CREATE INDEX idx_holidays_month  ON public.holidays(fixed_month, fixed_day);
```

```
CREATE INDEX idx_holidays_pub    ON public.holidays(is_published);
```

D. Table : holiday_submissions

```
-- ═══════════════════════════════════════════════════
-- TABLE: holiday_submissions
-- ═══════════════════════════════════════════════════
CREATE TYPE submission_status AS ENUM (
  'pending', 'under_review', 'approved', 'rejected'
);

CREATE TABLE public.holiday_submissions (
  id               UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  -- Auteur
  user_id          UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  -- Contenu soumis
  name             TEXT NOT NULL,
  proposed_date    DATE NOT NULL,
  category         holiday_category NOT NULL,
  description      TEXT,
  official_source TEXT NOT NULL,   -- URL obligatoire
  source_type      TEXT CHECK (source_type IN ('url', 'journal_officiel', 'communique',
'autre')),
  -- Workflow
  status           submission_status NOT NULL DEFAULT 'pending',
  reviewed_by      UUID REFERENCES public.profiles(id),
  reviewed_at      TIMESTAMPTZ,
  reviewer_note    TEXT,
  -- Si approuvé, lien vers le jour férié créé
  holiday_id       UUID REFERENCES public.holidays(id),
  -- Anti-spam
  ip_address       INET,
  -- Timestamps
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_submissions_user   ON public.holiday_submissions(user_id);
CREATE INDEX idx_submissions_status ON public.holiday_submissions(status);
CREATE INDEX idx_submissions_date   ON public.holiday_submissions(proposed_date);

-- Rate limiting : max 5 soumissions / utilisateur / 24h
CREATE OR REPLACE FUNCTION check_submission_rate_limit()
RETURNS TRIGGER AS $$
BEGIN
  IF (SELECT COUNT(*) FROM public.holiday_submissions
      WHERE user_id = NEW.user_id
        AND created_at > now() - INTERVAL '24 hours') >= 5 THEN
    RAISE EXCEPTION 'Rate limit: maximum 5 soumissions par 24h';
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER enforce_submission_rate_limit
  BEFORE INSERT ON public.holiday_submissions
  FOR EACH ROW EXECUTE FUNCTION check_submission_rate_limit();
```

E. Tables : quizzes & quiz_questions

```
CREATE TABLE public.quizzes (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  holiday_id  UUID NOT NULL REFERENCES public.holidays(id) ON DELETE CASCADE,
  title       TEXT NOT NULL,
  points_max  INTEGER NOT NULL DEFAULT 100,
  is_active   BOOLEAN NOT NULL DEFAULT true,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TYPE question_type AS ENUM ('mcq', 'true_false', 'drag_drop');
```

```
CREATE TABLE public.quiz_questions (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  quiz_id         UUID NOT NULL REFERENCES public.quizzes(id) ON DELETE CASCADE,
  question_text   TEXT NOT NULL,
  question_type   question_type NOT NULL DEFAULT 'mcq',
  options         JSONB NOT NULL,
  -- Format: [{"id":"a","text":"Réponse A","is_correct":true}, ...]
  explanation     TEXT,           -- Explication affichée après la réponse
  points          INTEGER NOT NULL DEFAULT 33,
  order_index     SMALLINT DEFAULT 0,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE public.quiz_attempts (
  id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  quiz_id      UUID NOT NULL REFERENCES public.quizzes(id),
  user_id      UUID NOT NULL REFERENCES public.profiles(id),
  answers      JSONB NOT NULL,   -- {"question_id": "option_id", ...}
  score        INTEGER NOT NULL DEFAULT 0,
  score_pct    NUMERIC(5,2),     -- % score (0.00 - 100.00)
  completed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (quiz_id, user_id)      -- Un seul essai par quiz
);

CREATE INDEX idx_attempts_user ON public.quiz_attempts(user_id);
CREATE INDEX idx_attempts_quiz ON public.quiz_attempts(quiz_id);
```

F. Tables : badges & user_badges

```
CREATE TYPE badge_trigger AS ENUM (
  'action', 'points', 'streak', 'referral', 'submission'
);

CREATE TABLE public.badges (
  id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  slug          TEXT UNIQUE NOT NULL,   -- ex: "historien", "patriote"
  name          TEXT NOT NULL,
  description   TEXT,
  icon_url      TEXT,
  trigger_type  badge_trigger NOT NULL,
  trigger_value JSONB,
  -- ex: {"action": "read_holiday", "holiday_slugs": ["independance","liberation"]}
  -- ex: {"min_points": 500}
  -- ex: {"min_months": 3}
  points_reward INTEGER NOT NULL DEFAULT 0,
  is_active     BOOLEAN NOT NULL DEFAULT true,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE public.user_badges (
  id         UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id    UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  badge_id   UUID NOT NULL REFERENCES public.badges(id),
  earned_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (user_id, badge_id)
);

CREATE INDEX idx_user_badges_user ON public.user_badges(user_id);

-- Données initiales : catalogue des badges
INSERT INTO public.badges (slug, name, description, trigger_type, trigger_value,
points_reward) VALUES
  ('historien',    'Historien',   'Lu les fiches du 17 et 30 août',             'action',
'{"action":"read_holidays","count":2}',     50),
  ('patriote',     'Patriote',    'Consulté toutes les fêtes nationales',        'action',
'{"action":"read_national_all"}',           75),
  ('culturel',     'Culturel',    'Exploré 3 fiches de fêtes religieuses',       'action',
'{"action":"read_religious","count":3}',    50),
  ('quiz_master',  'Quiz Master', 'Réussi 10 quiz avec score >= 80%',            'points',
'{"min_score_pct":80,"min_quizzes":10}',   100),
```

```
  ('ambassadeur',  'Ambassadeur', '3 invitations WhatsApp acceptées',            'referral',
'{"min_referrals":3}',                      150),
  ('assidu',       'Assidu',      'Calendrier consulté 3 mois consécutifs',      'streak',
'{"min_months":3}',                          75),
  ('contributeur', 'Contributeur','Une soumission approuvée',                   'submission',
'{"min_approved":1}',                        100);
```

G. Tables : notifications & préférences

```
CREATE TYPE notification_type AS ENUM (
  'reminder_d1', 'reminder_d7', 'bridge_alert', 'quiz_call',
  'badge_earned', 'submission_approved', 'submission_rejected', 'cultural_fact'
);
CREATE TYPE notification_channel AS ENUM ('push', 'email', 'sms');
CREATE TYPE notification_status  AS ENUM ('queued', 'sent', 'failed', 'cancelled');

CREATE TABLE public.notifications (
  id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id      UUID REFERENCES public.profiles(id) ON DELETE CASCADE,
  -- NULL = broadcast à tous
  type         notification_type NOT NULL,
  channel      notification_channel NOT NULL DEFAULT 'push',
  title        TEXT NOT NULL,
  body         TEXT NOT NULL,
  data         JSONB,            -- payload arbitraire (holiday_id, badge_id…)
  status       notification_status NOT NULL DEFAULT 'queued',
  scheduled_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  sent_at      TIMESTAMPTZ,
  error_msg    TEXT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_notif_user      ON public.notifications(user_id);
CREATE INDEX idx_notif_scheduled ON public.notifications(scheduled_at) WHERE status =
'queued';

CREATE TABLE public.notification_preferences (
  user_id           UUID PRIMARY KEY REFERENCES public.profiles(id) ON DELETE CASCADE,
  push_enabled      BOOLEAN NOT NULL DEFAULT true,
  email_enabled     BOOLEAN NOT NULL DEFAULT true,
  reminder_d1       BOOLEAN NOT NULL DEFAULT true,
  bridge_alerts     BOOLEAN NOT NULL DEFAULT true,
  quiz_calls        BOOLEAN NOT NULL DEFAULT true,
  cultural_facts    BOOLEAN NOT NULL DEFAULT false,
  push_token        TEXT,       -- token OneSignal
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

H. Tables : user_streaks & audit_logs

```
CREATE TABLE public.user_streaks (
  user_id          UUID PRIMARY KEY REFERENCES public.profiles(id) ON DELETE CASCADE,
  current_streak   SMALLINT NOT NULL DEFAULT 0,   -- mois consécutifs actuels
  longest_streak   SMALLINT NOT NULL DEFAULT 0,   -- record personnel
  last_activity    DATE,                           -- dernière date de consultation
  updated at       TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ==================================================
-- TABLE: audit_logs (traçabilité admin)
-- ==================================================
CREATE TABLE public.audit_logs (
  id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  actor_id     UUID NOT NULL REFERENCES public.profiles(id),
  action       TEXT NOT NULL,
  -- ex: 'approve_submission', 'create_holiday', 'update_badge', 'ban_user'
  target_type  TEXT,            -- 'holiday', 'submission', 'user'…
  target_id    UUID,
  old_data     JSONB,           -- état avant modification
  new_data     JSONB,           -- état après modification
```

```
  ip_address   INET,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_actor  ON public.audit_logs(actor_id);
CREATE INDEX idx_audit_target ON public.audit_logs(target_type, target_id);
CREATE INDEX idx_audit_date   ON public.audit_logs(created_at DESC);
```

3.3 Row Level Security (RLS)

```
-- Activer RLS sur toutes les tables publiques
ALTER TABLE public.profiles              ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.holidays              ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.holiday_submissions   ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.quizzes               ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.quiz_questions        ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.quiz_attempts         ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.badges                ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.user_badges           ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.notifications         ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.audit_logs            ENABLE ROW LEVEL SECURITY;

-- Fonction utilitaire : rôle de l'utilisateur courant
CREATE OR REPLACE FUNCTION public.current_user_role()
RETURNS TEXT AS $$
  SELECT role FROM public.profiles WHERE id = auth.uid();
$$ LANGUAGE sql STABLE SECURITY DEFINER;

-- ── Policies: profiles ──────────────────────────
CREATE POLICY "Lecture publique des profils"
  ON public.profiles FOR SELECT USING (true);

CREATE POLICY "Mise à jour de son propre profil"
  ON public.profiles FOR UPDATE
  USING (auth.uid() = id)
  WITH CHECK (auth.uid() = id AND role = 'user');

-- ── Policies: holidays ──────────────────────────
CREATE POLICY "Lecture des jours fériés publiés"
  ON public.holidays FOR SELECT
  USING (is_published = true OR public.current_user_role() IN ('admin','editor'));

CREATE POLICY "Création/modification par admin ou éditeur"
  ON public.holidays FOR ALL
  USING (public.current_user_role() IN ('admin', 'editor'));

-- ── Policies: holiday_submissions ───────────────
CREATE POLICY "Lecture de ses propres soumissions"
  ON public.holiday_submissions FOR SELECT
  USING (auth.uid() = user_id OR public.current_user_role() IN ('admin','moderator'));

CREATE POLICY "Soumettre si authentifié"
  ON public.holiday_submissions FOR INSERT
  WITH CHECK (auth.uid() = user_id AND auth.uid() IS NOT NULL);

CREATE POLICY "Valider si admin ou modérateur"
  ON public.holiday_submissions FOR UPDATE
  USING (public.current_user_role() IN ('admin', 'moderator'));

-- ── Policies: quiz_attempts ─────────────────────
CREATE POLICY "Ses propres tentatives"
  ON public.quiz_attempts FOR ALL
  USING (auth.uid() = user_id);

-- ── Policies: notifications ─────────────────────
CREATE POLICY "Ses propres notifications"
  ON public.notifications FOR SELECT
  USING (user_id = auth.uid() OR user_id IS NULL);
```

```sql
-- ── Policies: audit_logs ──────────────────────────
CREATE POLICY "Lecture audit par admin uniquement"
  ON public.audit_logs FOR SELECT
  USING (public.current_user_role() = 'admin');
```

04 — ROADMAP & PLAN DE LANCEMENT

4.1 Phases de Développement

Phase	Durée	Livrables clés
PHASE 0 Setup	Semaine 1	Repo GitHub, Supabase projet, schéma BDD, domaine, design system Figma
PHASE 1 MVP Core	Semaines 2–5	Calendrier public + fiches jours fériés + Auth (email + Google) + Export iCal
PHASE 2 Communauté	Semaines 6–8	Formulaire de soumission + workflow admin (validation temps réel) + notifications email
PHASE 3 Gamification	Semaines 9–12	Quiz, badges, points, leaderboard, barre de Patriotisme, push notifications
PHASE 4 Croissance	Semaines 13–16	SEO, widget partageable, QR Codes, partenariats influenceurs, Analytics dashboard

4.2 Stack Technique Recommandée

Besoin	Outil choisi	Justification
Framework web	Next.js 14	SSR pour SEO, App Router, TypeScript natif
Base de données	Supabase (PostgreSQL)	BaaS open source, Realtime, Auth, Storage intégrés
Styling	Tailwind CSS + shadcn/ui	Rapide, cohérent, accessible
Email transactionnel	Resend	API simple, React Email pour les templates
Push Web	OneSignal	Gratuit jusqu'à 10k subscribers
Hébergement	Vercel	Intégration Next.js native, Edge Functions, CDN
Monitoring	Sentry + Vercel Analytics	Erreurs front/back + métriques Core Web Vitals

4.3 Structure du Dépôt GitHub

```
gabon-feries/
├── .github/workflows/    # CI/CD (lint, tests, deploy)
├── docs/                 # Documentation technique du projet
│   ├── brief.md
│   ├── database.md
│   └── api.md
├── supabase/
│   ├── migrations/       # SQL versionnés (000_init.sql…)
│   ├── functions/        # Edge Functions (notifications, badge-check…)
│   └── seed.sql          # Données initiales (jours fériés, badges)
├── src/
│   ├── app/              # Next.js App Router
│   │   ├── (public)/     # Routes publiques (/, /calendrier, /[date])
```

```
│   │   ├── (auth)/        # Connexion, inscription
│   │   ├── (dashboard)/   # Profil, badges, soumissions
│   │   └── admin/         # Interface administration (protégée)
│   ├── components/        # Composants React réutilisables
│   ├── lib/               # Clients Supabase, utilitaires
│   └── types/             # Types TypeScript générés depuis Supabase
├── public/
│   └── icons/             # Icônes des badges, drapeaux
└── README.md
```

4.4 Documentation Open Source

Chaque phase fera l'objet d'un article de documentation publique. Voici la structure recommandée :

1. Episode 0 — Pourquoi ce projet ? (contexte, problématique, vision)
2. Episode 1 — Architecture & choix techniques (Next.js + Supabase + Vercel)
3. Episode 2 — Schéma de base de données expliqué ligne par ligne
4. Episode 3 — Authentification avec Supabase Auth (email + Google OAuth)
5. Episode 4 — Workflow de soumission et validation temps réel (Supabase Realtime)
6. Episode 5 — Gamification : badges, points et quiz de zéro
7. Episode 6 — Notifications Push Web avec OneSignal + Edge Functions
8. Episode 7 — Growth Hacking : SEO, export iCal, widget partageable
9. Episode 8 — Interface d'administration CMS sans code